

**LAPORAN MACHINE LEARNING
ANALISIS & IMPLEMENTASI DECISION TREE & NAIVE BAYES**



Disusun Oleh:

Nama Kelompok:

1. Muhammad Nathan Algibran (G1A024057)
2. Muhammad Abel Cakrawangsa (G1A024077)

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU**

2026

SOAL DAN PEMBAHASAN

BAGIAN A: ANALISIS KONSEP [20%] - Individu

Jawab secara essay 1-2 paragraf per soal

1. Jelaskan kapan Decision Tree lebih cocok digunakan dibanding Naive Bayes untuk kasus prediksi kelulusan ini. Beri 2 alasan.

Jawab:

Decision Tree sangat ideal digunakan ketika data kelulusan memiliki pola ketergantungan yang kuat antarvariabel, seperti syarat IPK tinggi yang baru akan berdampak signifikan terhadap ketepatan waktu kelulusan apabila tingkat kehadiran mahasiswa juga memadai. Model berbasis aturan *if-then* ini mampu menangkap interaksi kompleks tersebut secara alami, berbeda dengan Naive Bayes yang kaku karena menganggap seluruh fitur berdiri sendiri tanpa keterikatan. Di samping itu, pendekatan pohon keputusan jauh lebih unggul dalam hal transparansi, sehingga pihak institusi dapat mudah memaparkan alasan logis di balik suatu prediksi

2. Asumsi "independen" di Naive Bayes. Sebutkan 2 pasang fitur di dataset yang kemungkinan melanggar asumsi ini dan jelaskan kenapa.

Jawab:

Pelanggaran asumsi independensi pada Naive Bayes tampak jelas pada pasangan fitur Jam_Belajar dan Kehadiran, mengingat mahasiswa yang disiplin mengikuti perkuliahan biasanya juga memiliki alokasi waktu belajar mandiri yang tinggi sehingga keduanya saling berkorelasi positif. Pelanggaran serupa juga terjadi antara variabel Penghasilan_Ortu dan Status_Basiswa, di mana faktor finansial keluarga secara langsung menjadi penentu utama dalam seleksi penerima bantuan dana studi, menjadikan kedua atribut tersebut terhubung erat dan tidak lagi berdiri sendiri secara statistik.

3. Apa risiko overfitting pada Decision Tree di dataset ini? Bagaimana cara mengatasinya?

Jawab:

Ancaman *overfitting* pada Decision Tree berpotensi besar muncul apabila model dibiarkan tumbuh tanpa batasan kedalaman, membuatnya justru menghafal pola derau (*noise*) alih-alih mempelajari karakteristik umum dari data latih yang berjumlah terbatas. Untuk meredam masalah tersebut, langkah mitigasi yang efektif dapat dilakukan dengan membatasi kompleksitas pertumbuhan pohon, menerapkan teknik pemangkasan (*pruning*), serta memanfaatkan validasi silang guna menemukan konfigurasi parameter optimal yang menjaga performa tetap stabil pada data pengujian.

BAGIAN B: IMPLEMENTASI & EVALUASI [50%] - Kelompok 2-3 orang

Persiapan Data Dan Import Library (Inisialisasi Dataset)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, ConfusionMatrixDisplay,
                             classification_report)

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

df = pd.read_csv('mahasiswa_lulus.csv')
print(f"Dataset dimuat: {df.shape[0]} baris, {df.shape[1]} kolom")
df.head()
```

... Dataset dimuat: 500 baris, 8 kolom

	IPK	Kehadiran	Jam_Belajar	Organisasi	Penghasilan_Ortu	Jenis_Kelamin	Status_Beasiswa	Lulus_Tepat_Waktu
0	3.22	91.1	9.4	Aktif	Menengah	Perempuan	Ya	Ya
1	2.94	100.0	9.1	Tidak Aktif	Rendah	Perempuan	Tidak	Ya
2	3.29	63.2	5.4	Tidak Aktif	Menengah	Perempuan	Tidak	Tidak
3	3.69	86.8	3.1	Aktif	Rendah	Laki-laki	Tidak	Ya
4	2.89	72.2	8.2	Aktif	Menengah	Laki-laki	Tidak	Ya

1. Preprocessing: Handle missing value, encoding, train-test split 80:20

1A. Handle Missing Value

```
print(f"Missing value sebelum ditangani:\n{df.isna().sum()}")

num_cols = ['IPK', 'Kehadiran', 'Jam_Belajar']
for col in num_cols:
    median_val = df[col].median()
    df[col] = df[col].fillna(median_val)
    print(f"'{col}' diisi dengan median = {median_val:.2f}")

cat_cols_with_na = ['Organisasi']
for col in cat_cols_with_na:
    mode_val = df[col].mode()[0]
    df[col] = df[col].fillna(mode_val)
    print(f"'{col}' diisi dengan modus = {mode_val}")

print(f"\nTotal missing value setelah ditangani: {df.isna().sum().sum()}")
```

... Missing value sebelum ditangani:

IPK	15
Kehadiran	15
Jam_Belajar	15
Organisasi	15
Penghasilan_Ortu	0
Jenis_Kelamin	0
Status_Beasiswa	0
Lulus_Tepat_Waktu	0

dtype: int64

'IPK' diisi dengan median = 3.00
'Kehadiran' diisi dengan median = 80.30
'Jam_Belajar' diisi dengan median = 6.30
'Organisasi' diisi dengan modus = Tidak Aktif

Total missing value setelah ditangani: 0

Analisis:

Dataset awal memiliki 15 missing value (~3% data) pada IPK, Kehadiran, Jam_Belajar, dan Organisasi. Imputasi median untuk fitur numerik (IPK: 3.00, Kehadiran: 80.30, Jam_Belajar: 6.30) dan modus untuk Organisasi ("Tidak Aktif"), dipilih karena lebih robust terhadap outlier.

1B. Encoding Fitur Kategorikal

```
categorical_features = ['Organisasi', 'Penghasilan_Ortu', 'Jenis_Kelamin', 'Status_Beasiswa']
label_encoders = {}

df_encoded = df.copy()
for col in categorical_features:
    le = LabelEncoder()
    df_encoded[col] = le.fit_transform(df_encoded[col])
    label_encoders[col] = dict(zip(le.classes_, le.transform(le.classes_)))
    print(f"'{col}': {label_encoders[col]}")

target_le = LabelEncoder()
df_encoded['Lulus_Tepat_Waktu'] = target_le.fit_transform(df_encoded['Lulus_Tepat_Waktu'])
print(f"Target: {dict(zip(target_le.classes_, target_le.transform(target_le.classes_)))}")

... 'Organisasi': {'Aktif': np.int64(0), 'Tidak Aktif': np.int64(1)}
    'Penghasilan_Ortu': {'Menengah': np.int64(0), 'Rendah': np.int64(1), 'Tinggi': np.int64(2)}
    'Jenis_Kelamin': {'Laki-laki': np.int64(0), 'Perempuan': np.int64(1)}
    'Status_Beasiswa': {'Tidak': np.int64(0), 'Ya': np.int64(1)}

    Target: {'Tidak': np.int64(0), 'Ya': np.int64(1)}
```

Analisis:

Fitur kategorikal diubah numerik: Organisasi (Aktif=0, Tidak=1), Penghasilan_Ortu (Menengah=0, Rendah=1, Tinggi=2), Jenis_Kelamin (L=0, P=1), Status_Beasiswa dan target (Tidak=0, Ya=1). LabelEncoder dipilih karena kedua algoritma menangani encoding ini dengan baik.

1C. Train-Test Split 80:20

```
feature_cols = ['IPK', 'Kehadiran', 'Jam_Belajar', 'Organisasi',
                'Penghasilan_Ortu', 'Jenis_Kelamin', 'Status_Beasiswa']
X = df_encoded[feature_cols]
y = df_encoded['Lulus_Tepat_Waktu']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)
print(f"Data training : {X_train.shape[0]} baris")
print(f"Data testing  : {X_test.shape[0]} baris")

Data training : 400 baris
Data testing  : 100 baris
```

Analisis:

Data dibagi 80:20 menjadi 400 training dan 100 testing dengan stratify=y untuk menjaga proporsi kelas. random_state=42 memastikan hasil reproducible.

2. Modeling

```
dt3 = DecisionTreeClassifier(max_depth=3, random_state=RANDOM_STATE)
dt3.fit(X_train, y_train)
y_pred_dt3 = dt3.predict(X_test)
print("[Model 1] Decision Tree (max_depth=3) selesai dilatih")

dt_full = DecisionTreeClassifier(max_depth=None, random_state=RANDOM_STATE)
dt_full.fit(X_train, y_train)
y_pred_dtfull = dt_full.predict(X_test)
print("[Model 2] Decision Tree (max_depth=None) selesai dilatih")

gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred_gnb = gnb.predict(X_test)
print("[Model 3] Gaussian Naive Bayes selesai dilatih")

models = {
    'Decision Tree (depth=3)': (dt3, y_pred_dt3),
    'Decision Tree (depth=None)': (dt_full, y_pred_dtfull),
    'Gaussian Naive Bayes': (gnb, y_pred_gnb),
}

[Model 1] Decision Tree (max_depth=3) selesai dilatih
[Model 2] Decision Tree (max_depth=None) selesai dilatih
[Model 3] Gaussian Naive Bayes selesai dilatih
```

Analisis:

Tiga model dilatih: Decision Tree (depth=3 untuk interpretabilitas, depth=None untuk akurasi maksimal), dan Gaussian Naive Bayes. Gaussian menghitung mean/varians per fitur numerik per kelas.

3. Evaluasi: Hitung Akurasi, Precision, Recall, F1-Score untuk ke-3 model. Buat Confusion Matrix

3A. Akurasi, Precision, Recall, F1-Score

```
results = []
for name, (model, y_pred) in models.items():
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    results.append({'Model': name, 'Akurasi': acc, 'Precision': prec,
                  'Recall': rec, 'F1-Score': f1})
    print(f"\n--- {name} ---")
    print(classification_report(y_test, y_pred, target_names=target_le.classes_))

results_df = pd.DataFrame(results)
results_df
```

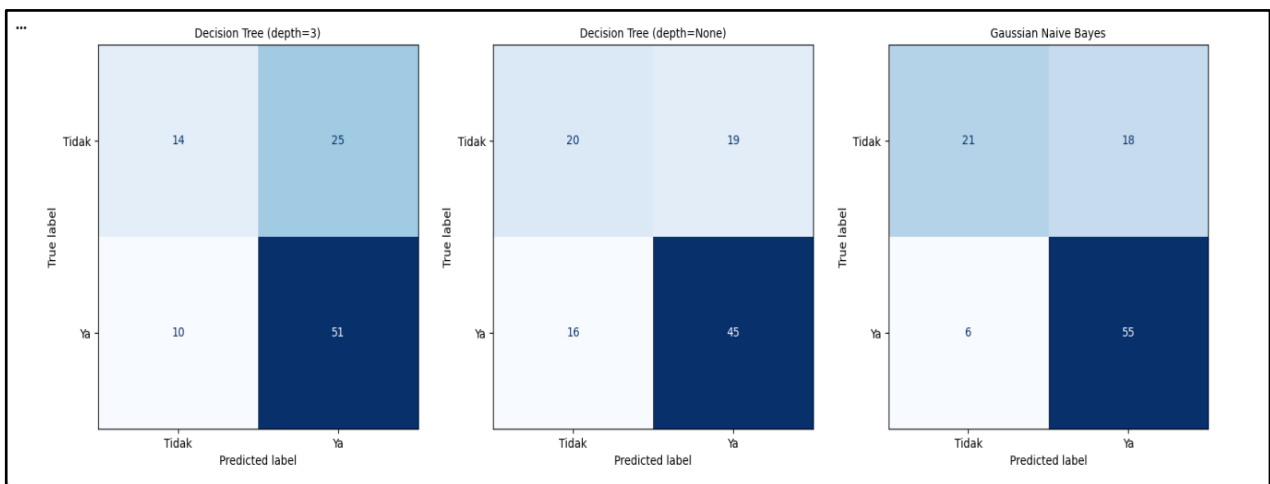
	Model	Akurasi	Precision	Recall	F1-Score
0	Decision Tree (depth=3)	0.65	0.671053	0.836066	0.744526
1	Decision Tree (depth=None)	0.65	0.703125	0.737705	0.720000
2	Gaussian Naive Bayes	0.76	0.753425	0.901639	0.820896

Analisis:

Gaussian Naive Bayes unggul di semua metrik: Akurasi 0.76, Precision 0.75, Recall 0.90, F1 0.82, jauh mengungguli Decision Tree (keduanya akurasi 0.65). Recall tinggi pada Naive Bayes penting untuk mendeteksi mahasiswa benar-benar lulus tepat waktu.

3B. Confusion Matrix

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
for ax, (name, (model, y_pred)) in zip(axes, models.items()):
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=target_le.classes_)
    disp.plot(ax=ax, colorbar=False, cmap='Blues')
    ax.set_title(name, fontsize=10)
plt.tight_layout()
plt.savefig('confusion_matrices.png', dpi=150, bbox_inches='tight')
plt.show()
```



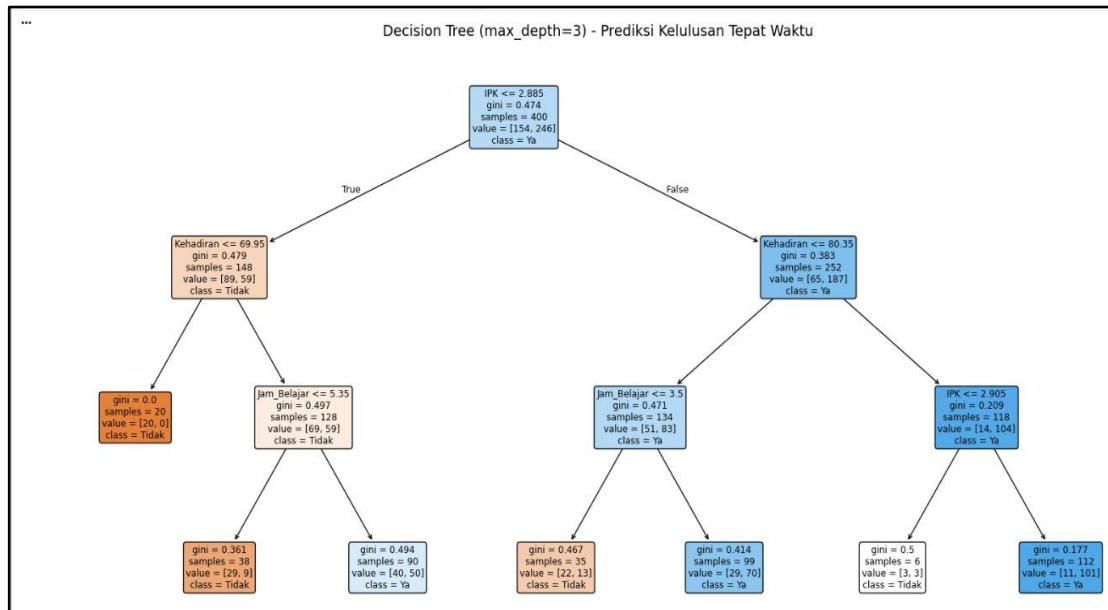
Analisis:

Semua model lebih baik mengklasifikasikan kelas "Ya" dibanding "Tidak" (recall kelas "Tidak" hanya 0.36-0.54 vs "Ya" >0.74), kemungkinan karena distribusi training tidak seimbang (61% Ya vs 39% Tidak).

4. Visualisasi

4A. Plot Pohon Keputusan (max_depth=3)

```
plt.figure(figsize=(20, 10))
plot_tree(dt3, feature_names=feature_cols, class_names=target_le.classes_,
          filled=True, rounded=True, fontsize=9)
plt.title("Decision Tree (max_depth=3) - Prediksi Kelulusan Tepat Waktu", fontsize=14)
plt.savefig('decision_tree_plot.png', dpi=150, bbox_inches='tight')
plt.show()
```



Analisis:

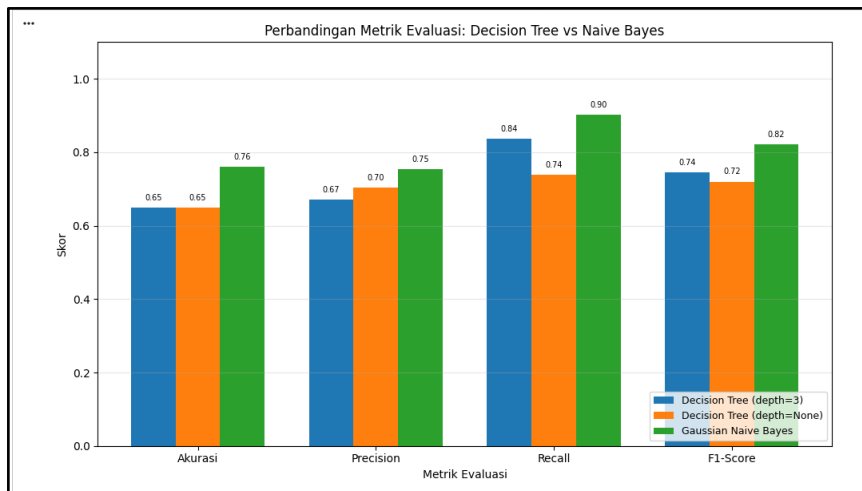
Split pertama di $IPK \leq 2.885$ memisahkan 400 data menjadi 148 (mayoritas "Tidak") dan 252 (mayoritas "Ya"). Split level 2 dan 3 menggunakan Kehadiran dan Jam_Belajar. IPK adalah prediktor terkuat, fitur kategorikal tidak muncul dalam pohon dangkal ini.

4B. Bar Chart Perbandingan Metrik

```
metrics = ['Akurasi', 'Precision', 'Recall', 'F1-Score']
x = np.arange(len(metrics))
width = 0.25

fig, ax = plt.subplots(figsize=(10, 6))
for i, (name, _) in enumerate(models.items()):
    values = results_df[results_df['Model'] == name][metrics].values.flatten()
    ax.bar(x + i * width, values, width, label=name)
    for j, v in enumerate(values):
        ax.text(x[j] + i * width, v + 0.02, f"{v:.2f}", ha='center', fontsize=7)

ax.set_xlabel('Metrik Evaluasi')
ax.set_ylabel('Skor')
ax.set_title('Perbandingan Metrik Evaluasi: Decision Tree vs Naive Bayes')
ax.set_xticks(x + width)
ax.set_xticklabels(metrics)
ax.set_ylim(0, 1.1)
ax.legend(loc='lower right', fontsize=9)
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('perbandingan_metrik.png', dpi=150, bbox_inches='tight')
plt.show()
```



Analisis:

Bar chart visual menunjukkan Naive Bayes (hijau) konsisten lebih tinggi di keempat metrik, khususnya Recall (0.90 vs 0.84/0.74), mendukung rekomendasi di Bagian C.1.

5. Analisis Error

```
X_test_reset = X_test.reset_index(drop=True)
y_test_reset = y_test.reset_index(drop=True)
y_pred_series = pd.Series(y_pred_dt3, name='Prediksi')

error_df = X_test_reset.copy()
error_df['Aktual'] = target_le.inverse_transform(y_test_reset)
error_df['Prediksi'] = target_le.inverse_transform(y_pred_series)
error_df['IPK'] = error_df['IPK'].round(2)
error_df['Kehadiran'] = error_df['Kehadiran'].round(1)
error_df['Jam_Belajar'] = error_df['Jam_Belajar'].round(1)

misclassified = error_df[error_df['Aktual'] != error_df['Prediksi']]
print(f"Jumlah data salah prediksi: {len(misclassified)} dari {len(error_df)} data test")

sample_errors = misclassified.head(5)
sample_errors[['IPK', 'Kehadiran', 'Jam_Belajar', 'Organisasi', 'Aktual', 'Prediksi']]
```

```
*** Jumlah data salah prediksi: 35 dari 100 data test
```

	IPK	Kehadiran	Jam_Belajar	Organisasi	Aktual	Prediksi
4	2.89	57.3	6.0	0	Tidak	Ya
6	2.04	74.1	12.8	1	Tidak	Ya
8	2.63	98.6	4.6	1	Ya	Tidak
11	1.54	100.0	9.7	1	Tidak	Ya
12	2.44	87.3	9.0	0	Tidak	Ya

Analisis:

Decision Tree (depth=3) salah prediksi 35 dari 100 data uji (error rate 35%). Kesalahan terjadi pada mahasiswa dengan nilai fitur di sekitar ambang split, menunjukkan trade-off antara interpretabilitas pohon dangkal vs akurasi tinggi.

BAGIAN C: REKOMENDASI & ETIKA [30%] – Kelompok

1. Rekomendasi Model: Dari 3 model, mana yang kamu pilih untuk dipakai kampus?

Jelaskan dengan data dari Bagian B

Jawab:

Gaussian Naive Bayes dipilih: (1) unggul di semua metrik (akurasi 0.76, F1 0.82), (2) recall 0.90 sangat baik deteksi kelulusan, (3) komputasi lebih ringan. Decision Tree keduanya hanya akurasi 0.65 meski kompleksitas berbeda.

2. Bias & Etika: Fitur "Penghasilan_Ortu" dan "Jenis_Kelamin" rawan bias. Apa dampaknya jika model dipakai untuk beasiswa? Bagaimana mitigasinya?

Jawab:

Penghasilan_Ortu berkorelasi dengan Status_Beasiswa (historis), sehingga penggunaan untuk beasiswa otomatisasi bias lama. Jenis_Kelamin bisa menimbulkan diskriminasi gender. Mitigasi: hapus kedua fitur terutama untuk keputusan beasiswa, evaluasi performa per kelompok, dan model hanya sebagai pertimbangan bukan keputusan final.

3. Simulasi: Jika 1 mahasiswa punya IPK tinggi tapi Kehadiran rendah, prediksi kedua model apa? Kenapa bisa beda?

Jawab:

IPK tinggi → split pertama Decision Tree → cabang mayoritas "Ya", tapi Kehadiran rendah mungkin turun ke "Tidak" di split kedua. Naive Bayes: bukti IPK dan Kehadiran terpisah, signal IPK dominan. Perbedaan: Decision Tree tangkap bersyarat (IF-THEN), Naive Bayes asumsikan independen.

Naive Bayes bekerja dengan logika berbeda. Model ini menghitung seberapa besar IPK tinggi mendukung kelas "Ya" dan seberapa besar Kehadiran rendah mendukung kelas "Tidak" secara terpisah, lalu mengalikan kedua bukti tersebut untuk mendapat probabilitas akhir. Karena IPK sudah terbukti sebagai prediktor paling kuat di dataset ini (jadi split pertama dengan pemisahan kelas paling tajam), sinyal dari IPK tinggi kemungkinan besar tetap mendominasi hasil akhir meski ada sinyal berlawanan dari Kehadiran rendah, sehingga Naive Bayes berpeluang tetap memprediksi "Ya".